

Pruebas de Seguridad ISO/IEC 25010 en KineSys

Guia paso a paso: herramienta, comando, subcaracteristica y resultado esperado.

Objetivo: que cualquier compañero pueda aplicar este flujo en su propio proyecto.

Regla para explicar: no basta con ejecutar comandos. Hay que decir que herramienta se usa, que parte del proyecto revisa, que subcaracteristica evalua y que significa el resultado.

Resumen rapido

Paso	Herramienta	Que revisa	Subcaracteristica ISO 25010
1	npm audit raiz	Dependencias JavaScript generales	Confidencialidad e integridad
2	npm audit frontend	Dependencias React/Vite	Confidencialidad e integridad
3	Jest	Login, token, servicios frontend	Autenticidad e integridad
4	Vite build	Compilacion del frontend	Apoyo a integridad del producto
5	ESLint	Errores de codigo frontend	Apoyo a integridad del producto
6	NuGet audit	Dependencias .NET	Confidencialidad e integridad
7	xUnit	Pruebas del backend	Autenticidad e integridad
8	PowerShell	Controladores sin autorizacion	Confidencialidad y autenticidad
9	PowerShell / Semgrep	Secretos JWT en codigo	Confidencialidad y autenticidad
10	PowerShell / Semgrep	Exposicion de Password	Confidencialidad
11	Swagger/Postman/PowerShell	Endpoint sin token	Confidencialidad y autenticidad
12	Auditorias	Registro de acciones	Responsabilidad y no repudio
13	OWASP ZAP	App/API ejecutandose	Confidencialidad, integridad y autenticidad

Comando inicial:

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys
```

Frase: Todas las pruebas se ejecutan desde el proyecto KineSys. Algunas revisan frontend, otras backend y otras seguridad general.

Paso 1 - Dependencias JavaScript de la raiz

Herramienta npm audit

Subcaracterística Confidencialidad e integridad

Por que aqui: se ejecuta en la raiz porque ahi hay un package.json general.

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys
```

```
npm audit --audit-level=moderate
```

Resultado esperado:

```
found 0 vulnerabilities
```

Que decir: Esta prueba evalua confidencialidad e integridad porque una libreria vulnerable podria filtrar datos o alterar el sistema. El resultado es correcto porque npm reporta cero vulnerabilidades.

Paso 2 - Dependencias del frontend

Herramienta npm audit

Subcaracterística Confidencialidad e integridad

Por que aqui: se ejecuta en kinesys porque ahi esta el frontend React/Vite.

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\kinesys
```

```
npm audit --audit-level=moderate
```

Resultado esperado:

```
found 0 vulnerabilities
```

Que decir: Revisamos las librerias del frontend porque este maneja formularios, token y datos de usuario. Si una dependencia es vulnerable, puede afectar confidencialidad o integridad.

Paso 3 - Pruebas del frontend

Herramienta Jest

Subcaracterística Autenticidad e integridad

Por que aqui: las pruebas del frontend validan login, contexto de autenticacion, token, cliente API y servicios.

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\kinesys
```

```
npm run test:jest -- --silent
```

Resultado esperado:

```
Test Suites: 10 passed, 10 total
Tests:      107 passed, 107 total
```

Que decir: Apoya autenticidad porque revisa partes relacionadas con login y token. Tambien apoya integridad porque confirma que los servicios siguen funcionando despues de las correcciones.

Paso 4 - Compilar frontend

Herramienta Vite

Subcaracterística No es prueba directa de seguridad; apoya integridad del producto

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\kinesys
```

```
npm run build
```

Resultado esperado:

```
✓ built
```

Nota: en esta version el build termina limpio, sin advertencias de SignalR ni de tamaño de chunks.

Que decir: Esta prueba confirma que las correcciones no rompieron la aplicacion. Por eso se usa como control de apoyo a la integridad.

Paso 5 - Lint del frontend

Herramienta ESLint

Subcaracterística No es prueba directa de seguridad; apoya integridad del producto

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\kinesys
```

```
npm run lint
```

Resultado esperado actual:

Sin errores ni advertencias

Que decir: ESLint ayuda a detectar errores de código. En este resultado la salida es limpia porque no imprime errores ni advertencias.

Paso 6 - Dependencias .NET del backend

Herramienta dotnet package list --vulnerable

Subcaracterística Confidencialidad e integridad

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\api
```

```
dotnet package list --project .\ApiPrueba.sln --vulnerable --include-transitive
```

Resultado esperado:

El proyecto "ApiPrueba" especificado no tiene paquetes vulnerables en los orígenes actuales.
El proyecto "ApiPrueba.Tests" especificado no tiene paquetes vulnerables en los orígenes actuales.

Que decir: Esta prueba revisa paquetes NuGet del backend. Evalúa confidencialidad e integridad porque el backend procesa datos sensibles y no debe depender de paquetes vulnerables.

Paso 7 - Pruebas del backend

Herramienta xUnit con dotnet test

Subcaracterística Autenticidad e integridad

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\api
```

```
dotnet test .\ApiPrueba.sln
```

Resultado esperado:

Resumen de pruebas: total: 39; con errores: 0; correcto: 39; omitido: 0

Que decir: Esta prueba apoya autenticidad porque valida reglas de usuarios, roles y token. También apoya integridad porque confirma que la lógica del backend sigue correcta.

Paso 8 - Controladores protegidos

Herramienta PowerShell

Subcaracterística Confidencialidad y autenticidad

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys
```

```
$controllers = Get-ChildItem api\ApiPrueba\Controllers -Filter *.cs
```

```
$without = @($controllers | Where-Object { (Get-Content -Raw $_.FullName) -notmatch '[Authorize|AuthorizeByRole' ]})
```

```
"Total controllers: $($controllers.Count)"
```

```
"Without auth attributes: $($without.Count)"
```

Resultado esperado:

```
Total controllers: 35  
Without auth attributes: 0
```

Que decir: Evalua confidencialidad porque evita endpoints publicos con datos sensibles. Evalua autenticidad porque obliga a que el usuario este identificado antes de acceder.

Paso 9 - Secretos JWT en el codigo

Herramienta PowerShell con Select-String

Subcaracterística Confidencialidad y autenticidad

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys
```

```
Get-ChildItem .\api\ApiPrueba -Recurse -Include *.cs,*.json |  
Select-String -Pattern 'ClaveDeAcceso_ParaAccederTokens_2025'
```

```
Select-String -Path .\api\ApiPrueba\appsettings.json -Pattern '"Key"'
```

Resultado esperado: no debe imprimir coincidencias.

Que decir: Evalua confidencialidad porque un secreto escrito en codigo puede filtrarse. Tambien evalua autenticidad porque con la clave JWT alguien podria intentar falsificar tokens.

Paso 10 - No devolver Password en respuestas

Herramienta PowerShell con Select-String

Subcaracterística Confidencialidad

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys
```

```
Get-ChildItem .\api\ApiPrueba\Controllers,.\api\ApiPrueba\Models -Recurse -Include *.cs |  
Select-String -Pattern 'p\.Password|Password,'
```

Resultado esperado: no debe mostrar p.Password en respuestas.

Que decir: Evalua confidencialidad porque una contraseña, incluso si esta hasheada, no debe devolverse en una respuesta de la API.

Paso 11 - Endpoint sin token

Herramienta PowerShell, Swagger o Postman

Subcaracterística Confidencialidad y autenticidad

Terminal 1:

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\api\ApiPrueba
```

```
dotnet run --launch-profile http
```

Terminal 2:

```
try {  
    Invoke-WebRequest -Uri http://localhost:5058/api/Citas -UseBasicParsing  
} catch {  
    $_.Exception.Response.StatusCode.value__  
}
```

Resultado esperado:

```
401
```

Que decir: 401 significa que el sistema rechazo una peticion sin token. Eso demuestra autenticidad, porque exige identidad, y confidencialidad, porque no deja ver datos a usuarios anonimos.

Paso 12 - Login y JWT

Herramienta Swagger, Postman o PowerShell

Subcaracterística Autenticidad

http://localhost:5058/swagger

POST /api/Personas/Login

Resultado esperado:

```
{
  "id": 1,
  "nombres": "...",
  "apellidos": "...",
  "token": "eyJ..."
}
```

Que decir: Esta prueba evalúa autenticidad porque confirma que el sistema valida credenciales y genera un token JWT firmado para acceder a recursos protegidos.

Paso 13 - Auditoria

Herramienta Swagger, Postman o base de datos

Subcaracterística Responsabilidad y no repudio

Prueba: iniciar sesión, crear/editar/eliminar un dato importante y consultar auditorías.

Campos esperados:

```
IdPersona
NombreUsuario
FechaHora
DireccionIP
Accion
Entidad
Exitoso
```

Que decir: Evalúa responsabilidad porque permite saber que usuario hizo una acción. Evalúa no repudio porque queda evidencia con usuario, fecha, IP, acción y resultado.

Paso 14 - OWASP ZAP

Herramienta OWASP ZAP

Subcaracterística Confidencialidad, integridad y autenticidad

Levantar API:

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\api\ApiPrueba
```

```
dotnet run --launch-profile http
```

Levantar frontend:

```
cd C:\Users\carde\Documents\ProgramacionMovil\KineSys\kinesys
```

```
npm run dev
```

En ZAP Desktop:

1. Abrir OWASP ZAP.
2. Ir a Quick Start.
3. Elegir Automated Scan.
4. Usar `http://localhost:5173`.
5. Para API, importar `http://localhost:5058/swagger/v1/swagger.json`.

Resultado esperado: sin alertas High o Critical.

Que decir: ZAP analiza la aplicacion desde afuera, como un atacante controlado. Revisa comportamiento real: headers, CORS, endpoints expuestos, XSS e inyecciones.

Flujo para explicarlo en clase

1. **Dependencias:** uso npm audit y NuGet audit. Subcaracterísticas: confidencialidad e integridad.
2. **Pruebas automaticas:** uso Jest y xUnit. Subcaracterísticas: autenticidad e integridad.
3. **Proteccion de endpoints:** uso PowerShell, Swagger o Postman. Subcaracterísticas: confidencialidad y autenticidad.
4. **Secretos y datos sensibles:** uso PowerShell o Semgrep. Subcaracterísticas: confidencialidad y autenticidad.
5. **Auditoria:** uso Swagger, Postman o base de datos. Subcaracterísticas: responsabilidad y no repudio.
6. **Análisis dinamico:** uso OWASP ZAP. Subcaracterísticas: confidencialidad, integridad y autenticidad.

Cierre corto

En seguridad ISO 25010 no se usa una sola herramienta para todo. Se combinan herramientas de dependencias, pruebas automaticas, revision de codigo, pruebas manuales de autenticacion, auditoria y analisis dinamico.

En KineSys los resultados principales fueron correctos: cero vulnerabilidades en npm, cero vulnerabilidades en NuGet, 107 pruebas frontend aprobadas, 39 pruebas backend aprobadas, 35 controladores protegidos y cero controladores sin autorizacion.